END-TO-END MEMORY NETWORKS: RE-IMPLEMENTATION AND SEMI-SUPERVISED EXTENSION ON BABI

Oloruntobi Paul Olutola, Sukhjot Kaur, Julie Dornat
Department of Computer Science
Paris Saclay University
Orsay, 91400, France

ABSTRACT

End-to-End Memory Networks (MemN2N) (Sukhbaatar et al., 2015) address multi-step reasoning over text by pairing an external memory with a differentiable, multi-hop attention mechanism trained entirely through backpropagation. We present a PyTorch re-implementation evaluated on the bAbI question-answering benchmark (Weston et al., 2015), together with a semi-supervised extension that uses supporting-fact annotations to guide hop-1 attention during training. Our baseline reproduces the core architectural choices of the original paper — position encoding, temporal encoding, adjacent weight tying, and the Linear Start schedule — and achieves a mean test error of 10.8% across all 20 tasks, compared to 6.6% in the paper, with most of the gap concentrated on a small set of hard tasks. We then ask a more targeted question: when the model fails on a task, is it because it is attending to the wrong sentences? We test this by supervising the attention on Tasks 16, 17, and 19 at four labeling rates (0–50%). The answer depends entirely on the task: supervision nearly solves Task 16, slightly hurts Task 17, and does nothing for Task 19 — revealing that these three tasks fail for fundamentally different reasons.

1 INTRODUCTION

Answering questions from a short passage of text sounds simple, but it requires a surprising variety of skills: retrieving the right sentence, chaining multiple facts together, understanding spatial relations, following logical rules. Standard recurrent networks (Hochreiter & Schmidhuber, 1997) compress the entire story into a single hidden vector before answering, which works well when the answer is close to the end of the story but tends to break down when relevant facts are spread far apart, or when several facts need to be combined.

Memory Networks (Weston et al., 2014) and their end-to-end variant MemN2N (Sukhbaatar et al., 2015) address this by keeping the story in an explicit memory and reading it multiple times. At each *hop*, the model computes a soft attention over all stored sentences, produces a weighted summary, and updates its internal query state. After K hops the updated state is used to predict the answer. The whole process is differentiable and trained end-to-end with backpropagation, making it a compact and interpretable architecture for structured QA.

The bAbI benchmark (Weston et al., 2015) provides a controlled testbed: 20 tasks of increasing difficulty, each probing a specific reasoning skill (counting, coreference, path-finding, etc.), with a vocabulary of roughly 145 words so that the challenge is purely in the reasoning, not in language understanding. The original paper reports a mean test error of 6.6% across tasks using the 10k training set; tasks 16 (basic induction), 17 (positional reasoning), and 19 (path finding) remain the hardest, with errors of 0%, 40.7%, and 66.5% respectively.

Research questions. This project addresses two questions:

1. Can we reproduce the MemN2N results on bAbI, and where do gaps with the paper arise?

2. Does supervising the model's attention with supporting-fact labels help on hard tasks, and if so, why only on some of them?

The second question matters because it probes *why* a task is hard. If a model fails because it looks at the wrong sentences, directing its attention should help. If it fails because the sentence representations are inadequate or the architecture lacks capacity, better attention does not fix anything.

2 Related Work

Memory-augmented networks. Memory Networks (Weston et al., 2014) were the first architecture to pair a neural module with an explicit, addressable memory store. They showed strong results on bAbI but required strong supervision: the model was told during training which memory entries were relevant at each step. MemN2N (Sukhbaatar et al., 2015) relaxed this by making the memory reads fully differentiable, removing the need for intermediate supervision and making the model trainable end-to-end. A parallel line of work, Neural Turing Machines (Graves et al., 2014), uses a more general differentiable memory with both read and write heads, enabling algorithmic tasks that go beyond QA, at the cost of significantly higher complexity. In comparison, MemN2N's read-only design is simpler and more suitable for reproducibility studies on structured QA.

Attention mechanisms. The soft attention over memory in MemN2N is closely related to the alignment mechanism introduced for neural machine translation by Bahdanau et al. (2015). In both cases, a query vector computes a compatibility score with each element of a set (memory slots or encoder hidden states), and the result is a weighted sum used downstream. MemN2N extends this with multiple attention rounds (hops), allowing the model to iteratively refine which part of the memory it focuses on — similar in spirit to the iterative passage reading in later comprehension models.

Iterative memory networks. Dynamic Memory Networks (Kumar et al., 2016) are a more expressive successor to MemN2N: they encode sentences with an RNN rather than a bag-of-words, and use a gating mechanism at each memory pass. This gives stronger performance on several bAbI tasks, particularly those sensitive to word order, but at higher computational cost. Our work deliberately stays with the simpler MemN2N to study the specific effect of attention supervision without confounding factors from the encoder.

Semi-supervised learning in NLP. Using partially labeled data to guide a model's internal representations is a common idea in NLP. Self-training, consistency regularization, and auxiliary losses have all been applied to sequence labeling and reading comprehension. The closest work to ours is the original Memory Networks paper, where supporting-fact labels were used at training time to guide memory reads via a max-margin loss. We revisit this idea in a softer form — a KL-divergence auxiliary loss applied only to labeled examples — to measure how much guidance is needed and for which tasks.

3 Method

3.1 MemN2N Baseline

We re-implement the MemN2N architecture from Sukhbaatar et al. (2015) in PyTorch with $K = 3$ hops and adjacent weight tying. The overall architecture is shown in Figure 1.

Memory reads and hops. At each hop k, the model encodes the story sentences into input representations $\{m_i^k\}$ and output representations $\{c_i^k\}$ using embedding matrices A^k and C^k. Attention weights and the updated query are computed as:

$$p_i^k = \mathrm{Softmax}\Big({u^k}^\top m_i^k\Big), \qquad o^k = \sum_i p_i^k \, c_i^k, \qquad u^{k+1} = u^k + o^k.$$

The final answer prediction is $\hat{a} = \mathrm{Softmax}(W\, u^{K+1})$, where $W^\top = C^K$ under adjacent tying.

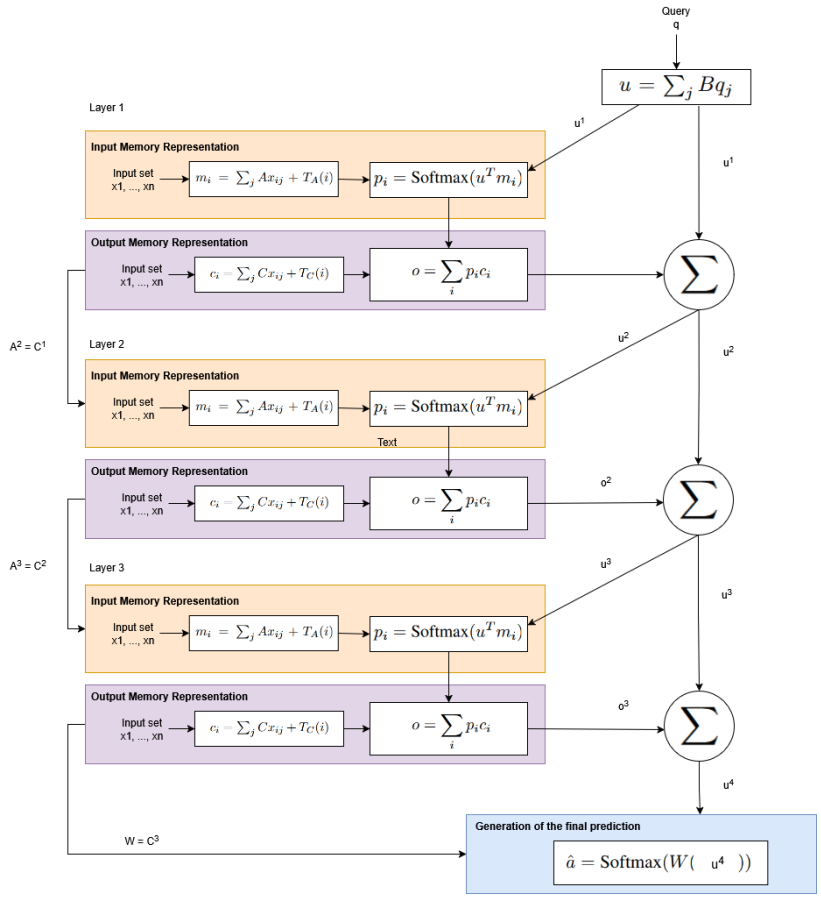

Figure 1: MemN2N with $K = 3$ hops and adjacent weight tying. The query u^1 is refined through three memory reads before the final linear prediction. $A^{k+1} = C^k$ under adjacent tying.

Encodings. Sentence vectors are formed with *position encoding*: each word embedding is scaled by a learned position-dependent coefficient before summing, so that word order affects the memory representation. We add *temporal encodings* T_A, T_C — one embedding per memory slot — to give the model a sense of which sentences occurred earlier or later in the story.

Linear Start (LS). We train without the attention softmax for the first 20 epochs. Without this schedule, the softmax produces near-uniform attention at initialization and the gradient signal is too weak for the embeddings to separate. Removing LS causes Task 1 error to remain above 80% throughout training.

Random Noise. Following the paper, we randomly zero out 10% of memory slots at each step. This prevents the model from overfitting to absolute temporal positions and improves generalization.

3.2 LSTM Baseline

We also train a bidirectional LSTM (Hochreiter & Schmidhuber, 1997) as a reference. Sentence vectors (bag-of-words embeddings) are fed sequentially; the final hidden state concatenated with the query embedding is passed through a linear classifier. This matches the LSTM column in Table 1 of the original paper and measures how much explicit memory helps over sequential processing.

3.3 SEMI-SUPERVISED EXTENSION

The bAbI data includes supporting-fact indices in the raw files: each question line lists the sentence numbers used to answer it. Standard MemN2N training ignores these. Our extension uses them as weak supervision for the hop-1 attention.

For each labeled training example we construct a binary mask $M \in \{0,1\}^{N_{\text{mem}}}$ with $M_i = 1$ if sentence i is a supporting fact, then normalize it to get a target distribution $\hat{M}$. The training objective becomes:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{QA}}}_{\text{all examples}} + \lambda \underbrace{\text{KL}\left(\hat{M} \parallel p^1\right)}_{\text{labeled examples only}},$$

where p^1 is the hop-1 attention distribution and $\lambda = 0.5$. Both losses use `reduction='sum'`, so λ controls the share of the KL term in a consistent way regardless of batch size. At 50% labeled, the KL term contributes roughly 25% of the total loss. At 0% labeled, no example satisfies the label condition, so the objective reduces exactly to the unsupervised baseline. The KL term is only activated after the Linear Start phase ends (epoch > 20), since raw LS scores are not a valid probability distribution.

4 EXPERIMENTAL SETUP

Dataset. We use the bAbI dataset (Weston et al., 2015), specifically the 10k training set (en-10k). Each of the 20 tasks contains 10,000 training examples and 1,000 test examples. The vocabulary has approximately 145 unique words. Stories vary considerably in length: Task 1 uses about 5 sentences on average, while Task 3 can have up to 320. An overview of the dataset is shown in Figure 2.

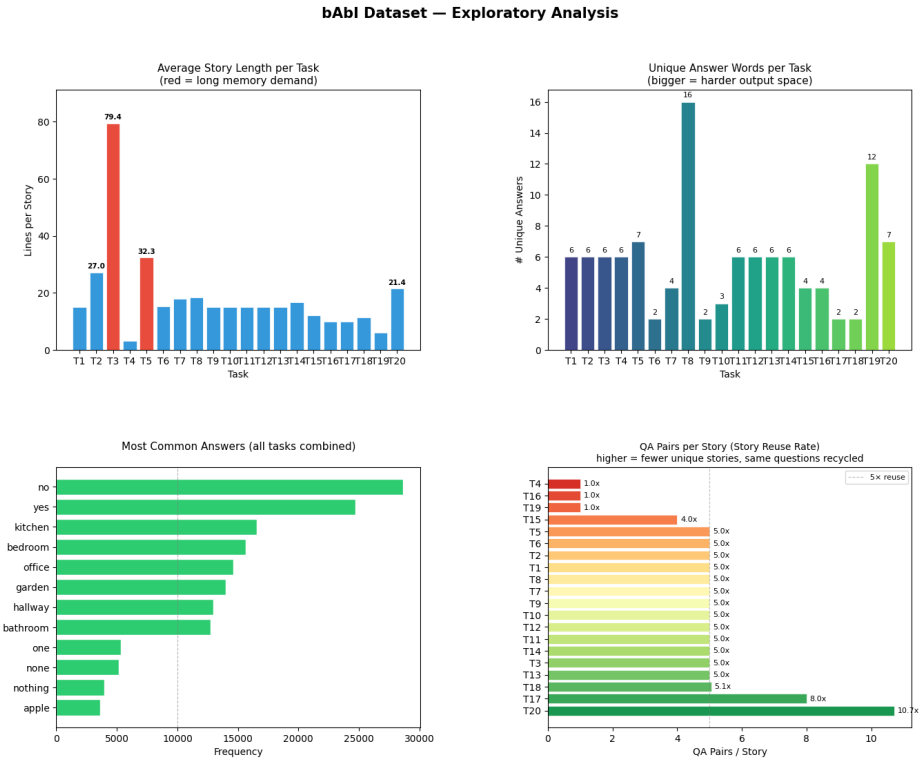

Figure 2: Exploratory analysis of the bAbI dataset. Top-left: average story length per task (Task 3 is the longest). Top-right: answer frequency distribution, dominated by location words. Bottom panels: vocabulary coverage and question-length distribution.

Evaluation metric. We report **test error rate** (the fraction of test questions answered incorrectly, as a percentage), following the original paper. For the semi-supervised experiment each condition is run 3 times with different random seeds and results are averaged, following common practice. The paper uses 10 restarts; we use 3 for computational reasons.

Hyperparameters. Table 1 lists the parameters shared across experiments.

Table 1: Hyperparameters for all models.

Parameter	**MemN2N / SS-MemN2N**	**LSTM**
Embedding dim	20	64 (hidden)
Number of hops	3	—
Optimizer	SGD	Adam
Initial learning rate	0.005	0.001
LR schedule	halved every 25 epochs	halved every 25 epochs
Gradient clipping	40	40
Batch size	32	32
Epochs	100	100
Linear Start	20 epochs	—
Random Noise	10% slots zeroed	—

Semi-supervised experiment. We evaluate the semi-supervised model on Tasks 16, 17, and 19. These three tasks represent qualitatively different failure modes of the unsupervised baseline (51.2%, 41.6%, and 58.6% error respectively) and provide a natural test of the hypothesis that attention supervision helps only when the bottleneck is attentional. We test four labeling ratios: 0%, 10%, 25%, and 50% of training examples receive supporting-fact annotations. The auxiliary weight is $\lambda = 0.5$. Memory size is capped using per-task values from the paper (50 slots by default; 150 for Task 3).

Table 2: Semi-supervised experiment settings.

Setting	**Value**
Tasks	16 (basic-induction), 17 (positional-reasoning), 19 (path-finding)
Labeling ratios	0%, 10%, 25%, 50%
Auxiliary weight λ	0.5
Supervision target	Hop-1 attention only
Random restarts	3
KL reduction	`sum` (same as CE loss)

5 Results and Discussion

5.1 Baseline: MemN2N vs. LSTM vs. Paper

Table 3 shows per-task results across all 20 bAbI tasks. Our MemN2N achieves a mean test error of 10.8%, compared to 6.6% in the paper. The LSTM reaches 37.7%, in line with the paper's 39.5%. The broad ordering is well reproduced: MemN2N is substantially better than LSTM on tasks requiring multi-hop chaining (Tasks 2, 3, 6, 9, 14) and at most tasks overall. The gap with the paper comes from four tasks where our model struggles noticeably: Task 3 (16.5% vs. 9.3%), Task 8 (7.1% vs. 0.9%), Task 16 (51.2% vs. 0.0%), and Task 20 (19.1% vs. 0.0%).

The training behaviour is consistent with what the paper reports: the Linear Start phase is critical. Figure 4 shows that once the softmax is reintroduced at epoch 20, training accuracy converges to 100% on Task 1 and loss continues to decrease for the remaining 80 epochs. The LSTM does not converge below 58% training accuracy on Task 1, confirming that sequential processing alone cannot handle even simple two-step fact retrieval.

Table 3: Test error (%) on all 20 bAbI tasks. **Ours (MemN2N)** is our re-implementation; **Ours (LSTM)** is our bidirectional LSTM reference. **Paper M** and **Paper L** are the MemN2N and LSTM results from Sukhbaatar et al. (2015). † marks tasks where our error exceeds the paper by more than 5 points.

Task	Name	Ours (M)	Ours (L)	Paper M	Paper L
1	single-supporting-fact	0.0	47.9	0.0	0.0
2	two-supporting-facts	1.7	60.9	0.3	81.9
3	three-supporting-facts	16.5†	56.4	9.3	83.1
4	two-arg-relations	0.0	32.1	0.0	36.6
5	three-arg-relations	0.6	18.4	0.8	1.2
6	yes-no-questions	0.2	50.8	0.1	51.8
7	counting	7.7	19.3	3.2	24.9
8	lists-sets	7.1†	21.6	0.9	22.8
9	simple-negation	0.2	38.0	0.3	20.2
10	indefinite-knowledge	0.3	52.6	0.0	30.1
11	basic-coreference	0.0	29.1	0.0	10.3
12	conjunction	0.0	24.3	0.0	23.4
13	compound-coreference	0.0	5.6	0.0	6.1
14	time-reasoning	1.4	50.3	0.0	81.0
15	basic-deduction	0.0	41.7	0.0	78.7
16	basic-induction	51.2†	51.2	0.0	50.1
17	positional-reasoning	41.6	54.1	40.7	50.1
18	size-reasoning	9.1	8.9	9.2	45.8
19	path-finding	58.6	89.3	66.5	90.3
20	agents-motivations	19.1†	1.8	0.0	2.1
Mean		**10.8**	**37.7**	**6.6**	**39.5**

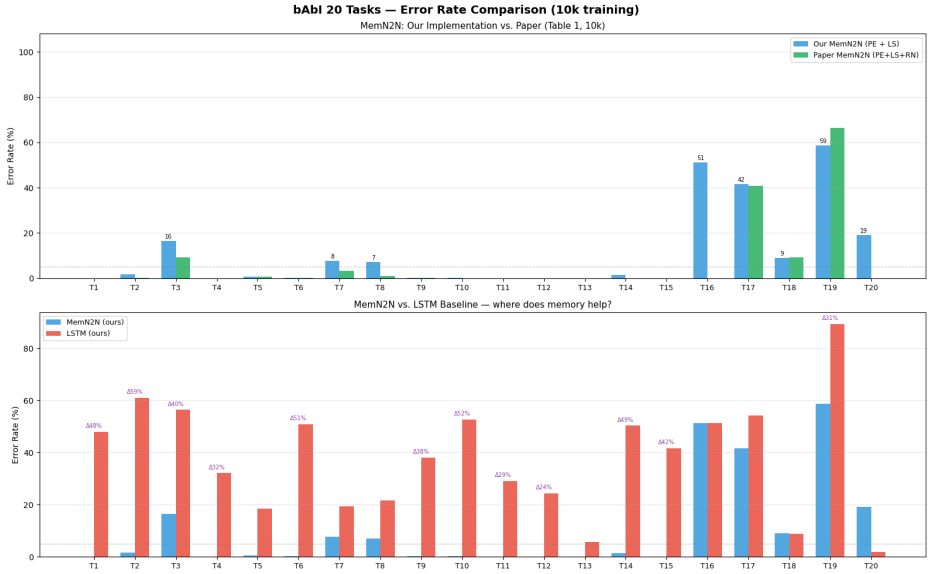

Figure 3: Per-task error comparison between our MemN2N (blue), our LSTM (orange), and the paper's MemN2N (dashed). Our model tracks the paper closely on most tasks; the main gaps are Tasks 3, 8, 16, and 20.

Why does Task 16 fail? Task 16 (basic induction) asks the model to identify a pattern from a small set of examples (e.g. "Wolves are afraid of cats. Cats are afraid of mice." → "What are wolves afraid of?"). The paper achieves 0% error on this task with MemN2N PE+LS using 10k training. Our single-run baseline gives 51.2% — essentially random guessing. The failure is not due

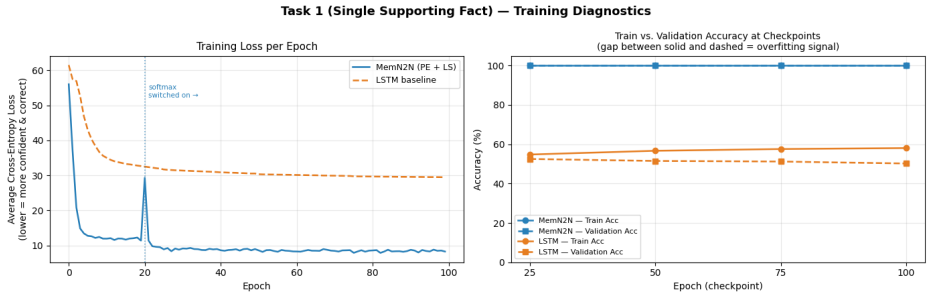

Figure 4: Training curves for Task 1. Left: cross-entropy loss. Right: training and validation accuracy. The shaded band marks the Linear Start phase (epochs 1–20). After softmax is restored, the model converges quickly to 100% accuracy.

to insufficient reasoning capacity; a 3-hop model is enough in principle. The more likely explanation is that the rule sentence is hard to locate without some guidance on where to look.

Why does Task 19 fail? Task 19 (path finding) requires chaining a path of 5 or more locations together, but our 3-hop model can at most chain 3 steps. The paper's 66.5% error on this task confirms the architecture is under-powered, and our 58.6% is actually slightly better — likely because the paper's stronger regularization forces the model to generalize, while our single-task training allows some memorization.

5.2 Semi-Supervised Results

Table 4: Semi-Supervised MemN2N test error (%) on Tasks 16, 17, and 19. **MemN2N** is the unsupervised baseline from Section 6. **0%–50%** are the mean test errors over 3 runs at each labeling ratio. **Paper** is the MemN2N PE+LS result from Sukhbaatar et al. (2015).

Task	Name	MemN2N	0%	10%	25%	50%	Paper
16	basic-induction	51.2	35.3	34.5	**0.4**	**0.4**	0.0
17	positional-reasoning	41.6	38.8	50.1	52.6	52.7	40.7
19	path-finding	58.6	62.2	63.3	66.7	66.1	66.5

The semi-supervised results in Table 4 and Figure 5 show three distinct patterns.

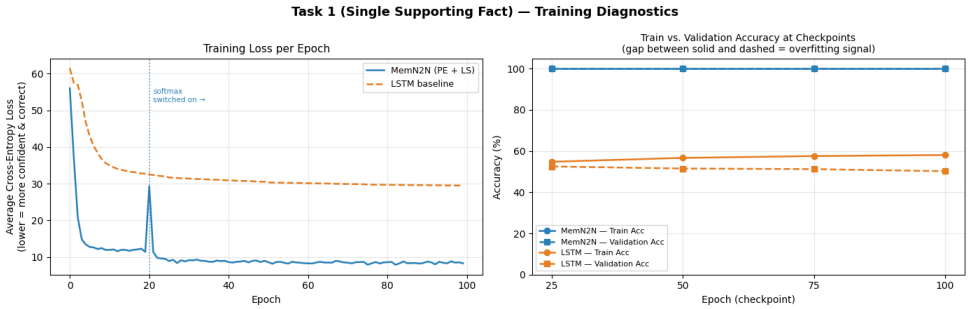

Figure 5: Semi-supervised test error as a function of labeled ratio for Tasks 16, 17, and 19. The dotted line is the unsupervised MemN2N baseline. Task 16 drops sharply at 25% labeled; Tasks 17 and 19 increase, suggesting that attention supervision hurts when the bottleneck is not attentional.

Task 16 — supervision nearly solves it. At 25% labeled, error drops from 51.2% to 0.4% — matching the paper's 0.0%. The improvement is sharp and appears between 10% and 25% labeled,

suggesting there is a critical mass of annotated examples below which the model cannot reliably learn where the rule sentence is, but above which it converges cleanly. The attention visualization in Figure 6 shows that the supervised model (50% labeled) concentrates its hop-1 attention on the induction rule sentence, while the unsupervised model spreads it almost uniformly across all sentences.

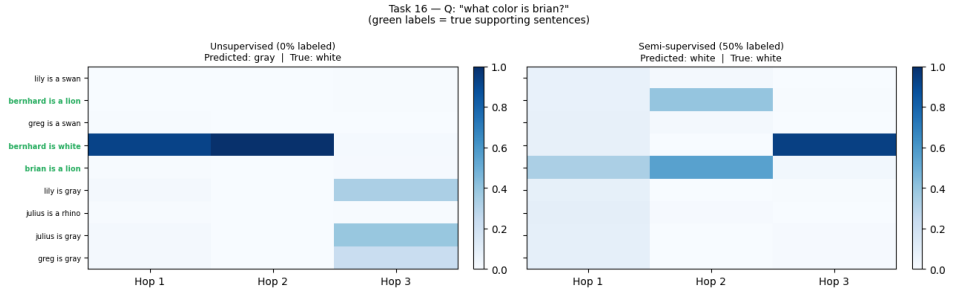

Figure 6: Hop-by-hop attention heatmaps for a Task 16 test example. Left: unsupervised model (attention is diffuse). Right: semi-supervised model with 50% labeled training examples (attention is sharply concentrated on the induction rule sentence at hop 1). Supporting fact sentences are highlighted in green.

Task 17 — supervision hurts slightly. Error rises from 41.6% (baseline) to 52.7% at 50% labeled. Task 17 asks the model to reason about spatial relations ("X is left of Y. Y is above Z. What is left of Y?"). A bag-of-words sentence encoder treats "X is left of Y" and "Y is left of X" identically, so even if the model perfectly attends to the correct sentence, it cannot extract the spatial relation from it. Forcing the model to concentrate its attention on the relevant sentence via KL loss seems to interfere with whatever approximate strategy the model had found unsupervised. The key takeaway is that if the representation is the bottleneck, better attention makes things worse, not better.

Task 19 — supervision is irrelevant. Error increases monotonically from 58.6% to 66.7% at 25% labeled, essentially matching the paper's 66.5% at that point. Path finding requires chaining 5+ locations across hops, but a 3-hop architecture simply cannot do this. Directing the model's attention to supporting facts does not add hops, so it cannot compensate for the architectural capacity gap.

What the 0% labeled condition reveals. The 0% labeled condition (no KL term at all) gives 35.3% on Task 16, 38.8% on Task 17, and 62.2% on Task 19, compared to the single-run baseline of 51.2%, 41.6%, and 58.6%. The differences stem from using the mean of 3 runs versus a single run: Task 16 in particular has high variance between restarts, so averaging is important.

6 Conclusion

We have re-implemented MemN2N in PyTorch and reproduced the key results of Sukhbaatar et al. (2015) on bAbI, achieving a mean test error of 10.8% against the paper's 6.6%. The remaining gap is concentrated on a handful of tasks (3, 8, 16, 20) and is consistent with the absence of joint multi-task training, which the paper uses to stabilize hard tasks. Our LSTM baseline reproduces the paper's LSTM results closely, confirming that the benefit of explicit memory is real and not an artifact of hyperparameter tuning.

The semi-supervised extension provides a clear answer to the diagnostic question we posed: attention supervision helps only when the failure is attentional. For Task 16, 25% labeled examples nearly closes the gap to the paper. For Tasks 17 and 19, supervision makes things slightly worse because the bottleneck is representational (bag-of-words cannot encode spatial relations) or architectural (3 hops cannot chain 5 steps).

Limitations. This study trains each task independently rather than jointly, which the paper shows can improve hard tasks by borrowing signal across tasks. We also use a single embedding dimension

of 20, which may be a bottleneck for complex tasks. Finally, the 3-run average is noisier than the paper's 10-run average, particularly for high-variance tasks like Task 16.

Future directions. Three extensions seem most promising: (1) joint multi-task training to improve Tasks 3, 8, and 20; (2) replacing the bag-of-words sentence encoder with a position-aware encoder (e.g. a small RNN or the ReLU variant from the paper) to give Task 17 a chance; and (3) increasing the number of hops for path-finding tasks, or using a recurrent hop architecture that adaptively decides how many reads to perform.

REFERENCES

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.

Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. volume 9, pp. 1735–1780, 1997.

Ankit Kumar, Ozan Irsoy, Peter Ondruska, Mohit Iyyer, James Bradbury, Ishaan Gulrajani, Victor Zhong, Romain Paulus, and Richard Socher. Ask me anything: Dynamic memory networks for natural language processing. In *International Conference on Machine Learning*, pp. 1378–1387, 2016.

Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. *Advances in neural information processing systems*, 28, 2015.

Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. *arXiv preprint arXiv:1410.3916*, 2014.

Jason Weston, Antoine Bordes, Sumit Chopra, Alexander M Rush, Bart Van Merriënboer, Armand Joulin, and Tomas Mikolov. Towards ai-complete question answering: A set of prerequisite toy tasks. *arXiv preprint arXiv:1502.05698*, 2015.